



المدرسة الوطنية للعلوم التطبيقية - آسفي
ⵜⴰⵎⴰⵔⵜ ⵜⴰⵏⴰⵢⵜ ⵜⴰⵖⴰⵏⴰⵏⵜ ⵜⴰⵙⴰⵍⴰⵏⵜ - ⵓⵙⴼⵉ
Ecole Nationale des Sciences Appliquées - Safi



École Nationale des Sciences Appliquées de Safi — Université Cadi Ayyad
Filière Génie Informatique de Données et Intelligence Artificielle (GIDIA)

RAPPORT DE FIN DE STAGE

Projet de Fin d'Année (PFA)

Effectué à

AGIRH

Présenté en vue de l'obtention du Diplôme d'Ingénieur d'État
Génie Informatique de Données et Intelligence Artificielle



Conception et réalisation d'un agent IA d'Onboarding et d'Offboarding RH

*Répondre aux questions RH par recherche documentaire sourcée
et informer sur l'avancement d'un dossier — sans jamais exécuter
d'action irréversible depuis la conversation.*

Réalisé par

Encadrant entreprise

M. Komla Edem Wilfried TSETSE M. Moulay Rachid Didi Alaoui

Dépôt du code (privé, accès sur demande) : github.com/wekt2k04/arhia

Architecture hexagonale · pipeline RAG multi-phases · orchestration conversationnelle sous garde-fou
anti-hallucination

Année universitaire 2025–2026

Remerciements

Je tiens à exprimer ma sincère reconnaissance à M. Moulay Rachid Didi Alaoui, mon encadrant au sein d'AGIRH, pour la confiance qu'il m'a accordée en me confiant un sujet exigeant et pour la liberté de conception qu'il m'a laissée tout en restant disponible à chaque étape décisive du projet.

Je remercie tout particulièrement M. Saad Khadir, AI and Software Engineer chez AGIRH, qui m'a encadré au quotidien tout au long de ce stage : sa disponibilité technique et ses retours exigeants ont directement façonné la qualité du travail présenté dans ce rapport.

Mes remerciements vont également à M. Mitar Issam, Directeur Général Adjoint d'AGIRH, pour son accueil et la confiance accordée à ce projet au sein de l'entreprise.

Je remercie également le corps professoral de l'École Nationale des Sciences Appliquées de Safi, et en particulier les enseignants de la filière Génie Informatique de Données et Intelligence Artificielle, pour la formation qui m'a permis d'aborder un sujet combinant ingénierie logicielle classique et intelligence artificielle appliquée avec les bases nécessaires.

Je remercie l'ensemble des personnes qui, au sein d'AGIRH, ont pris le temps de répondre à mes questions sur les processus réels d'onboarding et d'offboarding, sans lesquelles ce projet serait resté un exercice académique déconnecté d'un besoin concret.

Enfin, je remercie mes parents pour leur soutien constant tout au long de mes études et de ce stage.

Résumé

Ce rapport présente le travail réalisé durant un stage de fin d'année au sein de l'entreprise AGIRH, consistant à concevoir et réaliser **arhia**, un agent conversationnel dédié à l'onboarding et à l'offboarding des collaborateurs. Le projet repose sur une architecture hexagonale en .NET 8 et Next.js 15, un pipeline de génération augmentée par récupération (RAG) à quatre phases obligatoires (découpage structurel, embedding ONNX multilingue, indexation vectorielle Qdrant, reranking par cross-encodeur), et une couche d'orchestration conversationnelle à deux modèles (Router/Generator) protégée par un garde-fou anti-hallucination écrit en code plutôt que confié à une simple consigne de prompt.

Le système est vérifié de bout en bout : 245 tests automatisés (244 verts, un flake préexistant documenté), un déploiement Docker Compose fonctionnel sur base fraîche, et des réponses conversationnelles réellement sourcées, mesurées plutôt que supposées. Les limites connues — notamment un taux de mauvais classement du routeur d'intention d'environ 27 % avec le modèle local disponible — sont assumées et documentées comme telles.

Mots-clés : RAG, architecture hexagonale, LLM, garde-fou anti-hallucination, RBAC, ONNX Runtime, Qdrant, agent conversationnel RH.

Abstract

This report presents the work carried out during a final-year internship at AGIRH, consisting of designing and building **arhia**, a conversational agent dedicated to employee onboarding and offboarding. The project relies on a hexagonal architecture in .NET 8 and Next.js 15, a four-phase Retrieval-Augmented Generation (RAG) pipeline (structural chunking, multilingual ONNX embedding, Qdrant vector indexing, cross-encoder reranking), and a two-model conversational orchestration layer (Router/Generator) protected by an anti-hallucination guard implemented in code rather than left to prompt instructions alone.

The system is verified end-to-end : 245 automated tests (244 passing, one documented pre-existing flake), a working Docker Compose deployment on a fresh database, and conversational answers that are genuinely sourced, measured rather than assumed. Known limitations — notably an intent-routing misclassification rate of around 27 % with the currently available local model — are documented rather than hidden.

Keywords : RAG, hexagonal architecture, LLM, anti-hallucination guardrail, RBAC, ONNX Runtime, Qdrant, HR conversational agent.

Table des matières

Remerciements	2
Résumé	3
Abstract	3
Introduction générale	6
1 Contexte et cadrage du projet	7
1.1 L'entreprise d'accueil et le sujet de stage	7
1.2 Périmètre retenu et source de vérité métier	7
1.3 Rôles et organisation	8
1.4 Garde-fous métier de l'agent conversationnel	8
2 Architecture et choix techniques	9
2.1 Architecture hexagonale	9
2.2 Modèle de données	9
2.3 Justification des choix techniques	11
2.4 Sécurité : RBAC à portée et pattern BFF	12
2.5 Temps réel : un seul mécanisme pour deux usages	13
3 Le cœur du projet : pipeline RAG et orchestration conversationnelle	14
3.1 Pourquoi un RAG, et pourquoi un garde-fou	14
3.2 Le pipeline RAG en quatre phases obligatoires	14
3.3 Orchestration : Router et Generator	15
3.4 Le garde-fou anti-hallucination : deux portes de sortie en code	16
3.5 Limites mesurées et démarche empirique	16
4 Workflows métier	18
4.1 De l'employé au dossier réel : le flux d'onboarding	18
4.2 Circuit de validation qualité des templates	18
4.3 Cycle de vie et archivage	18
5 Interface utilisateur	19
5.1 Tableau de bord par rôle	19
5.2 Dossiers et collaborateurs	19
5.3 L'agent conversationnel en action	20
6 Résultats et vérifications	22
6.1 Tests automatisés	22
6.2 Vérifications de bout en bout	22
6.3 Évaluation du pipeline RAG	22

6.4	Déploiement	22
7	Difficultés rencontrées	23
7.1	Un décalage d'indices invisible entre deux tokenizers	23
7.2	Une optimisation par défaut qui casse un flux progressif	23
7.3	Un renommage de dossier refusé par Windows	23
7.4	Vérifier une palette de couleurs plutôt que de la juger à l'œil	23
8	Compétences acquises	25
8.1	Compétences techniques	25
8.2	Compétences méthodologiques	25
8.3	Compétences transversales	25
9	Bilan et perspectives	26
9.1	Ce qui est livré et vérifié	26
9.2	Limites assumées	26
9.3	Perspectives	26
	Conclusion générale	27
	Références	28
	Annexes	29

Introduction générale

L'intégration et le départ d'un collaborateur sont deux moments critiques de la vie d'une entreprise : ils mobilisent simultanément les ressources humaines, l'informatique et le management, et reposent aujourd'hui, dans de nombreuses organisations, sur des checklists suivies manuellement — avec le risque d'oubli que cela implique (un accès informatique non révoqué, un document non signé). C'est dans ce contexte qu'AGIRH, l'entreprise d'accueil de ce stage, a proposé un sujet consistant à concevoir un agent intelligent capable d'accompagner ces deux processus.

Ce rapport documente le travail réalisé en réponse à ce sujet : la conception et la réalisation d'**arhia**¹, un assistant conversationnel dédié à l'onboarding et à l'offboarding, appuyé sur un pipeline de recherche documentaire (RAG) et une orchestration conversationnelle par modèles de langage locaux. Le périmètre du projet a été délibérément resserré à ces deux processus — à l'exclusion des congés, de la paie ou de la gestion des temps — afin de livrer un socle solide plutôt qu'un prototype large mais fragile, une contrainte de périmètre qui s'est avérée, avec le recul, l'une des décisions les plus structurantes du stage.

Ce rapport s'organise en neuf chapitres. Le chapitre 1 présente le contexte du stage, le sujet confié et le périmètre métier retenu. Le chapitre 2 décrit l'architecture et les choix techniques. Le chapitre 3, le plus dense, détaille le cœur du système : le pipeline RAG et l'orchestration conversationnelle sous garde-fou. Le chapitre 4 présente les workflows métier et leur circuit de validation qualité. Le chapitre 5 présente l'interface utilisateur réalisée. Le chapitre 6 rassemble les résultats mesurés et les vérifications effectuées. Le chapitre 7 revient sur les difficultés techniques rencontrées, le chapitre 8 sur les compétences acquises, et le chapitre 9 dresse un bilan et ouvre sur les perspectives restantes, avant une conclusion générale.

1. Le nom du produit, *arhia*, est distinct du nom de l'entreprise d'accueil, AGIRH — une distinction volontaire, introduite en cours de stage pour ne pas confondre le prototype réalisé avec l'identité de l'entreprise.

1. Contexte et cadrage du projet

1.1 L'entreprise d'accueil et le sujet de stage

AGIRH est un éditeur marocain de logiciels de Gestion Intégrée des Ressources Humaines (SIRH), fondé par ISIS SOFT. Déployée dans plus de 400 entreprises au Maroc — dont environ 30 % des 500 plus grandes entreprises du pays, tous secteurs confondus (textile, maritime, pharmaceutique, assurance, automobile, agroalimentaire) — sa solution couvre l'ensemble des processus RH : administration du personnel, paie, organigramme, recrutement, formation, gestion des temps, évaluation, notes de frais et pilotage social par indicateurs. L'entreprise a récemment renforcé son expansion en Afrique via un partenariat avec Inetum.

Le stage s'est déroulé au sein d'AGIRH. Le sujet, tel que confié par l'entreprise, demandait la réalisation d'un *Agent IA d'Onboarding et d'Offboarding RH* : un assistant capable d'accompagner un nouveau collaborateur dès son intégration (présentation de l'entreprise, documents à fournir, accès aux outils, formations obligatoires, questions fréquentes) et de structurer le départ d'un collaborateur en fin de contrat, mobilité ou démission (restitution du matériel, clôture des accès, passation des dossiers, entretien de sortie). Le sujet mentionnait explicitement la possibilité de s'appuyer sur une base de connaissances RH, des workflows paramétrables, et une compréhension du langage naturel pour fournir des réponses contextualisées — sans pour autant en faire un périmètre fermé, laissant une part de conception au stagiaire.

Cette formulation ouverte a imposé un premier travail, avant tout code : cadrer précisément ce que le système devait faire, pour qui, et surtout ce qu'il ne devait **jamais** faire — une question centrale dès lors qu'un agent conversationnel touche à des données RH sensibles.

1.2 Périmètre retenu et source de vérité métier

Le périmètre a été volontairement resserré à l'onboarding et l'offboarding uniquement — sans congés/CET, sans paie, sans avance sur salaire. Ce choix n'est pas anodin : un périmètre plus large aurait dilué l'effort entre trop de processus à la fois, au détriment de la qualité de ce qui était réellement demandé.

La source de vérité métier retenue n'est pas une spécification abstraite, mais deux checklists qualité réelles de l'entreprise, au format SMSI (Système de Management de la Sécurité de l'Information, ISO 27001) : `SMSI.ENR.10-1` pour l'onboarding et `SMSI.ENR.10-2` pour l'offboarding. Les noms de personnes figurant dans ces documents réels (rédacteur, vérificateur, approbateur du template) ont été systématiquement anonymisés dans le code, la documentation et ce rapport.

Section (Onboarding)	Items représentatifs	Responsable
IT	Bitlocker, Kaspersky, filtrage web, agent GLPI, OneDrive	RH spécialisé IT
Messagerie	Double authentification, Office 365	RH spécialisé IT
Active Directory	Restrictions USB/exécutables, VLAN, synchro NTP	RH spécialisé IT
Sensibilisation SI	Politique mots de passe (10 car. min.), télétravail	RH du pôle
RH	Compte SELFRH, non-divulgateur, fiche de décharge	RH du pôle

L’offboarding suit la même logique en sens inverse (formatage du poste, suppression des accès, désactivation du profil AD, restitution). Un point de conception important : un seul workflow générique existe par type, quel que soit le motif de départ (fin de contrat, démission, mobilité, licenciement) — fidèle au document réel, qui ne distingue pas les motifs.

1.3 Rôles et organisation

Trois rôles structurent l’ensemble du système, à portée croissante : l’**Employee** (rôle par défaut à l’auto-inscription) ne voit que ses propres données ; le **RH**, un par pôle (département), crée les fiches de ses collaborateurs et joue le rôle de rédacteur dans le circuit de validation des templates ; l’**Admin/Qualité** a une portée globale et est seul habilité à élever le rôle d’un compte ou à valider un template. Il n’existe pas de rôle IT séparé : le travail informatique est porté par le RH “spécialisé IT” de chaque pôle, conformément à la structure réelle observée.

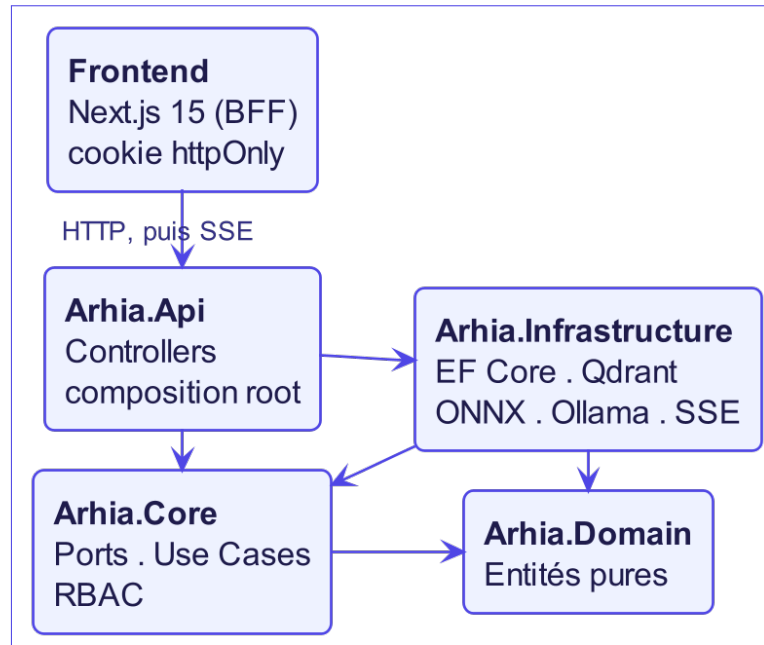
1.4 Garde-fous métier de l’agent conversationnel

Dès le cadrage, une règle non négociable a été posée : l’agent conversationnel est strictement **informatif**. Il ne déclenche **aucune action destructrice ou irréversible** depuis la conversation — ni suppression de compte, ni révocation d’accès, ni signature. Un statut de dossier peut être *lu*, jamais modifié depuis le chat. Cette contrainte, redétaillée au chapitre 3, n’est pas une simple précaution produite : sans garde-fou équivalent écrit en code, rien n’empêche un agent conversationnel de produire une réponse plausible mais non fondée — un risque particulièrement sensible dès lors que la conversation touche à des données RH.

2. Architecture et choix techniques

2.1 Architecture hexagonale

Le backend suit une architecture hexagonale (ports et adaptateurs) à quatre couches, avec une règle unique : une couche ne dépend que de celles listées en dessous d'elle, jamais l'inverse.



Architecture en couches : le frontend ne parle qu'à Arhia.Api, qui orchestre Core et Infrastructure sans jamais laisser la dépendance remonter vers Domain.

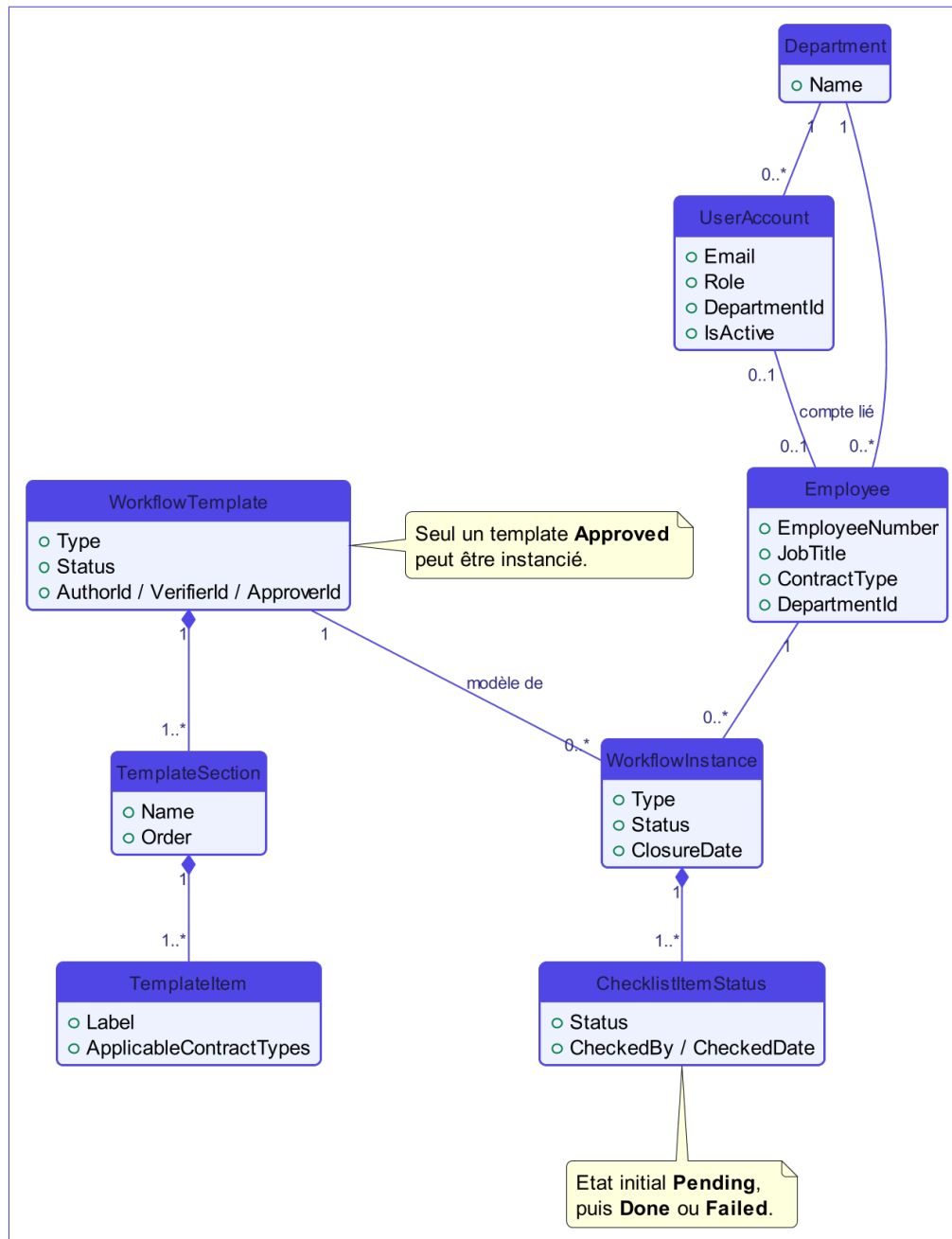
Arhia.Domain contient les entités pures — `Employee`, `Department`, `WorkflowTemplate`, `WorkflowInstance`, `UserAccount` — sans la moindre dépendance externe ; ce projet ne sait même pas qu'une base de données existe. **Arhia.Core** définit les ports (interfaces) et la logique métier elle-même : use cases, RBAC. **Arhia.Infrastructure** fournit un adaptateur concret par port — EF Core, Qdrant, ONNX Runtime, Ollama, SSE — sans aucune règle métier, uniquement de la traduction mécanique. **Arhia.Api** est la composition root : le seul endroit du projet qui décide quel adaptateur concret brancher derrière chaque port, dans `Program.cs`.

Le bénéfice concret de cette discipline s'est vérifié à plusieurs reprises pendant le stage : remplacer un composant d'infrastructure (par exemple, faire évoluer le modèle d'embedding) ne touche que la couche Infrastructure, et la logique métier se teste entièrement sans la moindre infrastructure réelle — au moyen de doubles de test (Moq), ce qui a permis de construire une suite de 245 tests sans jamais démarrer un serveur SQL ou un moteur ONNX pour valider une règle métier.

2.2 Modèle de données

Le modèle de données porte l'ensemble des règles métier du chapitre 1. Un `WorkflowTemplate` organise des `TemplateSection`, elles-mêmes composées de `TemplateItem` ; un `WorkflowInstance`

— le dossier concret d'un collaborateur — possède une collection de `CheckListItemStatus`, copie opérationnelle d'un item de template au moment de l'instanciation.



Modèle de données simplifié. Un *WorkflowInstance* n'est jamais construit à partir d'un template autre qu'**Approved** — contrainte imposée par le code, pas seulement documentée (voir §4).

Un détail d'implémentation mérite d'être souligné : le lien logique entre un `TemplateItem` et les `CheckListItemStatus` qu'il génère passe par un identifiant de traçabilité (`TemplateItemId`), et non par une clé étrangère EF Core configurée comme telle — un choix de mapping assumé, qui reflète le fait qu'une checklist instanciée est une *copie indépendante* du template au moment T, non un lien vivant vers lui.

2.3 Justification des choix techniques

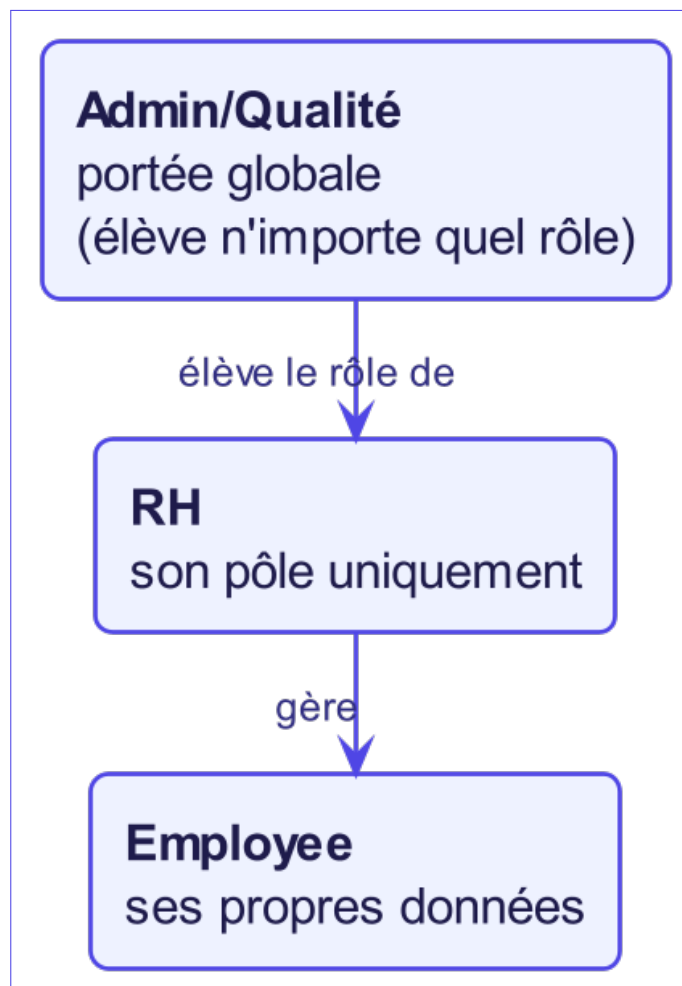
Les frameworks backend et frontend ont été conservés de la première version : une expertise déjà acquise n'apportait aucune valeur à réapprendre pour un projet de stage à périmètre resserré. En revanche, tout ce qui touche à l'intelligence artificielle et à l'infrastructure de données a été repensé intégralement, sans reprendre les décisions de la première version par défaut.

Composant	Choix et justification
Backend	.NET 8 (LTS), ASP.NET Core, EF Core — seul SDK installé au démarrage de l'implémentation, changement à coût faible si besoin
Frontend	Next.js 15, pattern BFF (Backend For Frontend) — le JWT n'est jamais exposé au navigateur
Authentification	JWT + ASP.NET Identity — Keycloak envisagé puis écarté comme disproportionné pour 3 rôles et une dizaine de comptes
Données relationnelles	SQL Server — réutilise l'infrastructure et l'expertise EF Core déjà en place
Données vectorielles	Qdrant, dédié — la valorisation d'une vraie compétence d'infrastructure IA prime sur la simplicité d'une solution embarquée
Chunking	Structurel (titres Markdown) avec recouvrement, via <code>Microsoft.ML.Tokenizers</code> (paquet officiel, WordPiece <i>et</i> SentencePiece)
Embedding	ONNX Runtime .NET pur, modèle multilingue 768d (<code>paraphrase-multilingual-mpnet-base-v2</code>) — corpus en français, contrôle total de la latence sans appel à Ollama pour cette étape
Reranking	Cross-encodeur ONNX BAAI/ <code>bge-reranker-v2-m3</code> , poids publiés directement — obligatoire , pas une amélioration optionnelle
LLM Router + Generator	Ollama local, <code>phi4-mini:3.8b</code> pour les deux rôles (écart documenté ci-dessous)
Temps réel	SSE (Server-Sent Events) — un seul mécanisme pour le chat en streaming et les notifications
Déploiement	Docker Compose, 4 services, Ollama natif sur l'hôte
Tests	xUnit + Moq + FluentAssertions

Écart assumé : un seul modèle pour deux rôles. L'intention initiale prévoyait un modèle distinct, plus capable, pour le Generator. En pratique, `gemma4:12b` — candidat naturel — a été testé réellement sur la machine de développement et n'a produit **aucune réponse en plus de deux minutes**, faute de GPU adapté. `phi4-mini:3.8b` répond en quelques secondes et reste la seule option praticable sur cette infrastructure pour les deux rôles. Ce compromis, documenté plutôt que dissimulé, est repris et approfondi au chapitre 3.

2.4 Sécurité : RBAC à portée et pattern BFF

Le contrôle d'accès repose sur deux mécanismes **distincts et vérifiés séparément**. Le premier, le RBAC, répond à la question « ce rôle peut-il faire cette action, en général? » ; toute la logique tient dans un dictionnaire figé (`RbacMatrix`), constante en code plutôt que donnée en base, donc jamais désynchronisée et trivialement testable.



Portée croissante des trois rôles : un Employee ne voit que ses propres données, un RH gère son pôle, seul l'Admin/Qualité a une portée globale et peut élever un rôle.

Le second mécanisme, `DepartmentScopeGuard`, répond à une question plus fine : « ce compte a-t-il le droit sur **cette** cible précise? ». Il est systématiquement vérifié **après** le RBAC, jamais à sa place : un RH qui cible un dossier hors de son pôle est refusé, quel que soit son rôle par ailleurs — une garantie vérifiée en conditions réelles pendant le stage (chapitre 6).

Enfin, le pattern **BFF** garantit que le navigateur ne détient jamais le JWT : le frontend Next.js appelle l'API côté serveur avec le jeton, puis pose un cookie `httpOnly`, invisible en JavaScript — invulnérable, en particulier, à un vol de jeton par script injecté (XSS).

2.5 Temps réel : un seul mécanisme pour deux usages

Le chat en streaming et la barre de notifications reposent tous deux sur **SSE** (Server-Sent Events) : un flux HTTP unidirectionnel serveur vers client, plus léger à mettre en œuvre que des WebSockets pour un besoin strictement descendant. Ce choix a révélé, en cours de développement, un piège instructif détaillé au chapitre 7 : une optimisation par défaut, invisible en apparence, peut casser silencieusement un flux progressif.

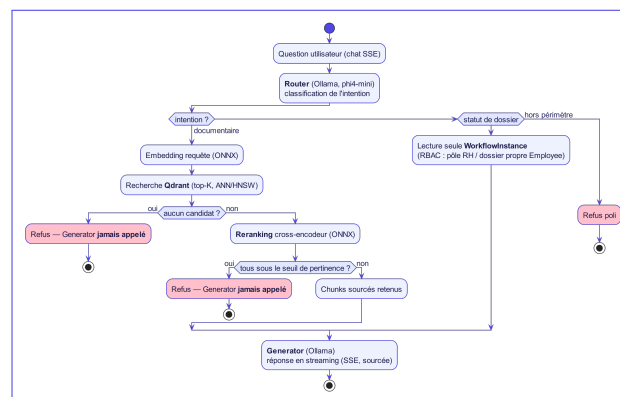
3. Le cœur du projet : pipeline RAG et orchestration conversationnelle

Ce chapitre décrit le sous-système le plus dense du projet, celui qui concentre la plus grande valeur ajoutée technique. Deux sous-systèmes distincts et complémentaires le composent : le **pipeline RAG**, qui répond à la question « où chercher l'information ? », et l'**orchestration conversationnelle**, qui répond à « que faire de la question ? ».

3.1 Pourquoi un RAG, et pourquoi un garde-fou

Un modèle de langage laissé à lui-même répond *de mémoire* : il peut produire une réponse plausible mais fausse (une hallucination), en particulier sur des politiques internes qu'il n'a évidemment jamais vues à l'entraînement. Le RAG (*Retrieval-Augmented Generation*) répond à ce problème en cherchant d'abord les passages réellement pertinents dans un corpus documentaire, puis en forçant le modèle à ne s'appuyer que sur ce contexte retrouvé. C'est précisément le risque pris au sérieux dès la conception de ce projet : sans reranking ni garde-fou explicite, rien ne garantit qu'une réponse « sourcée » le soit réellement.

3.2 Le pipeline RAG en quatre phases obligatoires



Les quatre phases du pipeline RAG, exécutées à l'ingestion (1–3) et à chaque question (2–4).

Phase 1 — Chunking structurel. Les documents sont découpés par structure (titres Markdown), pas par taille fixe aveugle, avec un recouvrement entre chunks voisins pour ne jamais perdre une information à cheval sur une frontière de découpage :

```
public MarkdownChunker(
    Func<string, int> countTokens,
    int maxTokensPerChunk = 400,
    double overlapRatio = 0.15)
```

Une section sous le budget (400 tokens par défaut) devient un chunk tel quel ; une section plus longue est sous-découpée paragraphe par paragraphe, avec 15 % de recouvrement conservé entre sous-chunks.

Phase 2 — Embedding. Chaque chunk devient un vecteur de 768 dimensions, via un modèle multilingue exécuté en ONNX Runtime .NET pur — aucun appel à Ollama pour cette étape, pour un contrôle total de la latence. La tokenisation elle-même a exigé un travail de correction non trivial : le tokenizer SentencePiece utilisé (`Microsoft.ML.Tokenizers`) produit des identifiants dans un espace différent de celui attendu par les poids ONNX publiés (convention Hugging Face) — un décalage vérifié empiriquement contre le fichier `tokenizer.json` du modèle, puis corrigé par une fonction de correspondance dédiée.

Phase 3 — Storage. Les vecteurs sont indexés dans **Qdrant**, en similarité cosinus, avec une recherche approximative (ANN, via HNSW) : on renonce à la garantie d'exactitude en échange d'une vitesse largement supérieure à grande échelle. Le corpus réel du projet — six documents — a été indexé et vérifié : 72 chunks au total.

Phase 4 — Reranking. Cette phase, la plus déterminante pour la qualité des réponses, distingue un pipeline RAG réellement fiable d'un RAG naïf. L'embedding (phase 2) est un **bi-encodeur** : il encode requête et documents *séparément*, rapide mais approximatif. Le reranking est un **cross-encodeur** : il encode la paire requête-document *ensemble*, en une seule passe — plus lent, nettement plus précis — selon le format standard de la famille RoBERTa (`<s> requête </s></s> document </s>`). D'où l'architecture à deux vitesses : le bi-encodeur réduit rapidement tout le corpus à quelques candidats, le cross-encodeur les réordonne finement.

Piège réel retenu. Un score de reranking élevé (jusqu'à 0,78 observé) ne garantit **pas** que le chunk contient la réponse — seulement qu'il est thématiquement proche. Le score mesure une proximité, pas une vérité : c'est pourquoi, comme le montre la section suivante, la décision finale relit le texte réellement généré, jamais le score seul.

3.3 Orchestration : Router et Generator

Deux rôles distincts se partagent deux appels à Ollama. Le **Router** classe chaque question en une intention parmi trois valeurs fermées : `DOCUMENTAIRE`, `STATUT_DOSSIER`, `HORS_PERIMETRE` — des libellés volontairement laissés en français dans le code, calibrés empiriquement avec le modèle. Sa sortie brute n'est **jamais** utilisée telle quelle :

```
if (normalized.Contains("STATUT_DOSSIER"))
    return ConversationIntent.CaseStatus;
if (normalized.Contains("DOCUMENTAIRE"))
    return ConversationIntent.DocumentaryQuestion;
return ConversationIntent.OutOfScope;
// vide, timeout, mot hallucine : jamais un acces elargi
```

Ce test par `.Contains(...)`, jamais par égalité stricte, tolère qu'un modèle de 3,8 milliards de paramètres ajoute du texte autour du mot attendu. Surtout : tout ce qui ne correspond exactement à aucune des deux premières valeurs — une sortie vide, un timeout réseau, un mot inventé — retombe par défaut sur `HORS_PERIMETRE`. C'est un système **fail-safe**, **pas fail-open** : une panne ou une ambiguïté du modèle ne peut jamais, par construction, élargir accidentellement un accès.

Le **Generator**, lui, ne reçoit que le contexte déjà filtré et écrit la réponse en streaming, fragment par fragment, via SSE.

3.4 Le garde-fou anti-hallucination : deux portes de sortie en code

C'est, de l'aveu même de l'expérience de ce stage, le point le plus important de tout le projet — et la réponse directe à l'échec silencieux constaté dans la première version. Deux portes de sortie anticipée existent **avant même** d'envisager d'appeler le Generator :

```
if (candidates.Count == 0)
    return new DocumentaryPreparation(
        false, null, Array.Empty<DocumentChunk>());
// ... reranking, filtrage par MinimumRelevanceThreshold = 0.01f ...
if (best.Count == 0)
    return new DocumentaryPreparation(
        false, null, Array.Empty<DocumentChunk>());
```

Si Qdrant ne retourne aucun candidat, ou si aucun ne passe le seuil de pertinence après reranking, le Generator n'est **jamais invoqué** : le système répond directement qu'il n'a pas trouvé l'information. Le seuil retenu (0,01) est volontairement bas : il sert de filtre de bruit, pas de garantie à lui seul — c'est le texte réellement généré qui est ensuite relu contre plusieurs formulations de refus possibles, pour détecter un refus déguisé même lorsque des chunks pertinents étaient disponibles. Ce n'est donc pas une consigne de prompt qu'un modèle pourrait ignorer sur une question ambiguë : c'est un garde-fou écrit en code, vérifiable, et couvert par des tests unitaires dédiés. C'est ce qui rend le mode de défaillance du système « gracieusement faux » — il peut se tromper de catégorie, mais ne ment jamais avec assurance sur un fait.

3.5 Limites mesurées et démarche empirique

Le routeur se trompe sur environ **27 %** des questions d'un jeu de test de 48 questions (13 sur 48 mal classées, en `STATUT_DOSSIER` ou `HORS_PERIMETRE` au lieu de `DOCUMENTAIRE`). Cette limite est mesurée et assumée, pas masquée. Une tentative d'amélioration — doubler les exemples few-shot dans le prompt du routeur — a été testée empiriquement puis **abandonnée** : reconfirmée sur 11 cas réels après coup, elle n'a corrigé qu'un seul cas sur onze, sans effet mesurable jugé suffisant, et la modification a été annulée plutôt que conservée sans preuve de bénéfice. Une seconde tentative, ajoutant une température basse au routeur dans l'espoir d'un comportement plus déterministe, a été mesurée à deux reprises (température 0,0 puis 0,15) sur le jeu de 48 questions complet : le score a **baissé** à chaque fois (25/48 contre une base de 30/48), résultat contre-intuitif mais cohérent avec l'hypothèse qu'un modèle de cette taille, n'ayant pas véritablement appris la règle de classification, devient déterministement faux sur les cas limites plutôt que parfois juste par hasard. Cette tentative a également été abandonnée.

Sur l'ensemble du pipeline conversationnel (routage puis recherche documentée puis génération), une mesure complète réalisée sur le jeu de 48 questions a produit 21 réponses

correctes, avant l'application de trois correctifs ciblés : (1) un indicateur **Sourced** qui restait à **true** même lorsque le générateur avait en réalité refusé en texte — un score de reranking élevé ne garantissant pas la présence de la réponse, comme déjà noté ; (2) une clarification du prompt du générateur pour réduire les refus injustifiés lorsque le contexte contenait pourtant la réponse ; (3) une correction de mots-clés trop stricts dans le jeu de questions lui-même. **Ces correctifs n'ont pas été reconfirmés sur l'ensemble des 48 questions** à la date de rédaction de ce rapport — une limitation assumée plutôt qu'un chiffre inventé pour paraître plus favorable.

Le vrai levier identifié pour améliorer le routage n'est ni le prompt, ni la température, mais la taille du modèle : un plan de bascule vers un serveur Ollama d'entreprise, avec des modèles plus capables (**phi4:14b** pour le routeur, **qwen3.5:9b** pour le générateur), a été préparé pour une présentation en fin de stage. Les résultats de ce test, mesuré en conditions réelles contre l'infrastructure de l'entreprise, ne sont pas encore consolidés dans la documentation du projet à la date de rédaction de ce rapport et ne sont donc pas présentés ici comme acquis.

4. Workflows métier

4.1 De l'employé au dossier réel : le flux d'onboarding

Créer une fiche collaborateur (poste, pôle, type de contrat, date) déclenche la résolution des items attendus contre le **dernier template approuvé** du type correspondant, croisé sur trois dimensions : poste $\times$ pôle $\times$ type de contrat. Un stagiaire, par exemple, ne reçoit typiquement pas de compte SELFRH ni de processus disciplinaire à signer — une personnalisation réellement appliquée par le code (`ResolveApplicableItems`), pas seulement documentée : sur le jeu de données de développement, les collaborateurs en contrat de stage reçoivent systématiquement moins d'items que leurs collègues en CDI (par exemple 20 items sur 22 en onboarding). Le `WorkflowInstance` ainsi créé porte une collection de `CheckListItemStatus` cochés indépendamment — chacun constituant une trace d'audit horodatable — jusqu'à clôture, puis archivage en lecture seule définitive.

4.2 Circuit de validation qualité des templates

Un template de checklist n'est jamais modifié librement : il suit un circuit à trois identités distinctes, reproduisant le processus qualité réel de l'entreprise.



*Un RH **propose**, un premier compte Admin/Qualité **vérifie**, un second **approuve** — trois identités distinctes, jamais la même personne deux fois.*

Cette contrainte de séparation des rôles est imposée **par le code de l'entité elle-même**, pas par une simple convention : la méthode `Approve` refuse explicitement qu'un vérificateur approuve son propre travail, et qu'un rédacteur approuve le sien. Surtout : **seul un template au statut Approved peut instancier un dossier réel** — une garantie vérifiée par le use case d'instanciation lui-même, pas seulement documentée dans ce rapport.

4.3 Cycle de vie et archivage

Une fois l'offboarding terminé (clôture signée), le dossier du collaborateur est archivé en lecture seule : consultable par RH et Admin/Qualité à des fins d'audit, mais plus jamais modifiable. Ce statut `Archived` est distinct d'un statut `Closed` intermédiaire, ce qui permet de distinguer, dans les données de développement, un dossier récemment clôturé d'un dossier définitivement clos — une distinction visible dans l'interface (chapitre 5).

5. Interface utilisateur

Le frontend suit le pattern BFF déjà décrit (§2.4) : chaque page authentifiée est servie par Next.js, qui relaie les appels vers l'API .NET côté serveur, sans jamais exposer le JWT au navigateur. L'interface reprend l'identité visuelle du produit — un indigo distinctif et un logo hexagonal, clin d'œil à l'architecture hexagonale réelle du système — disponible en mode clair et sombre, avec une palette sombre recalculée pour respecter un ratio de contraste WCAG AA plutôt qu'une simple inversion de couleurs.

5.1 Tableau de bord par rôle

Après connexion, le tableau de bord affiche un contenu adapté au rôle connecté : un Employee ne voit que son propre dossier, un RH voit les dossiers en cours de son pôle, un Admin/Qualité voit l'ensemble des pôles avec des compteurs agrégés. Une carte dédiée, visible uniquement pour ce dernier rôle, permet de déclencher la réindexation du corpus documentaire sans passer par une commande manuelle — une fonctionnalité ajoutée en cours de stage à la demande explicite de l'encadrant.

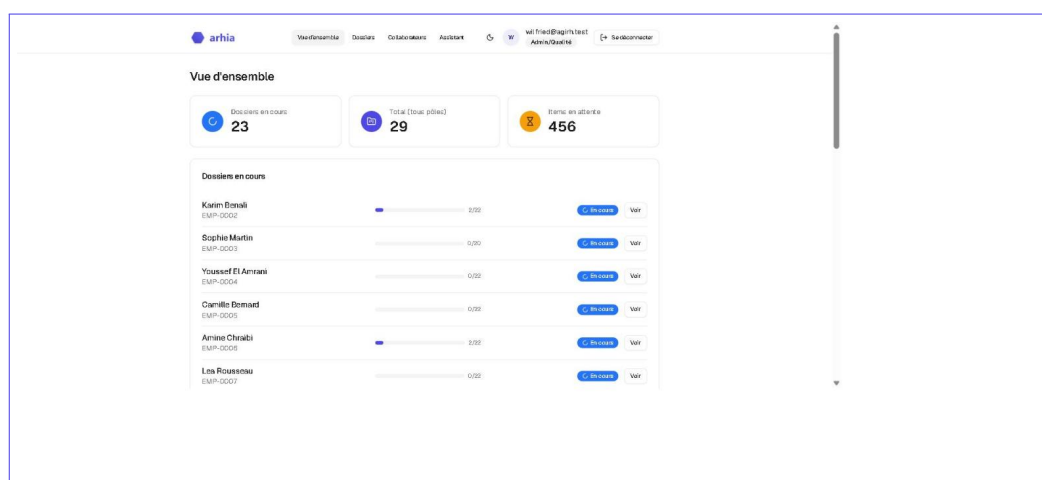
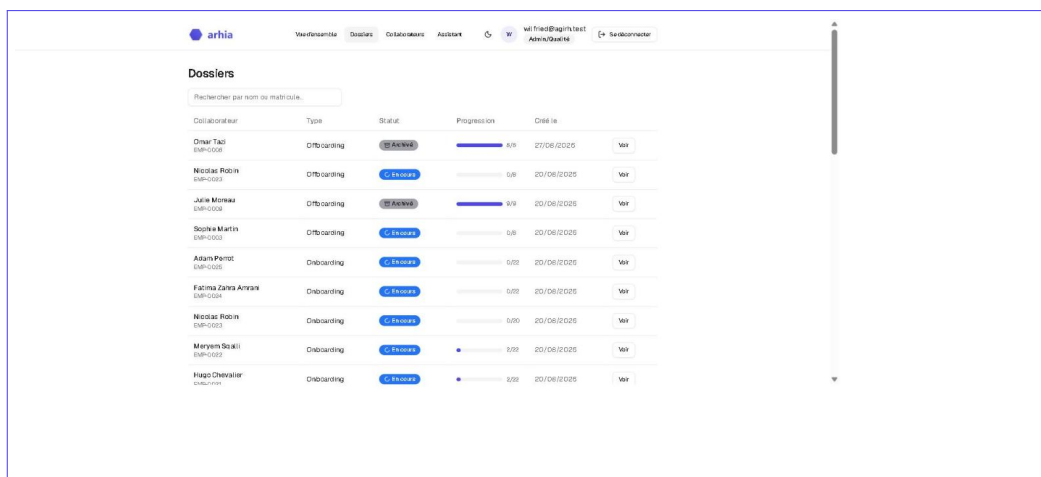


Tableau de bord, vue Admin/Qualité — 23 dossiers en cours, 29 au total, 456 items en attente, données réelles de l'environnement de développement.

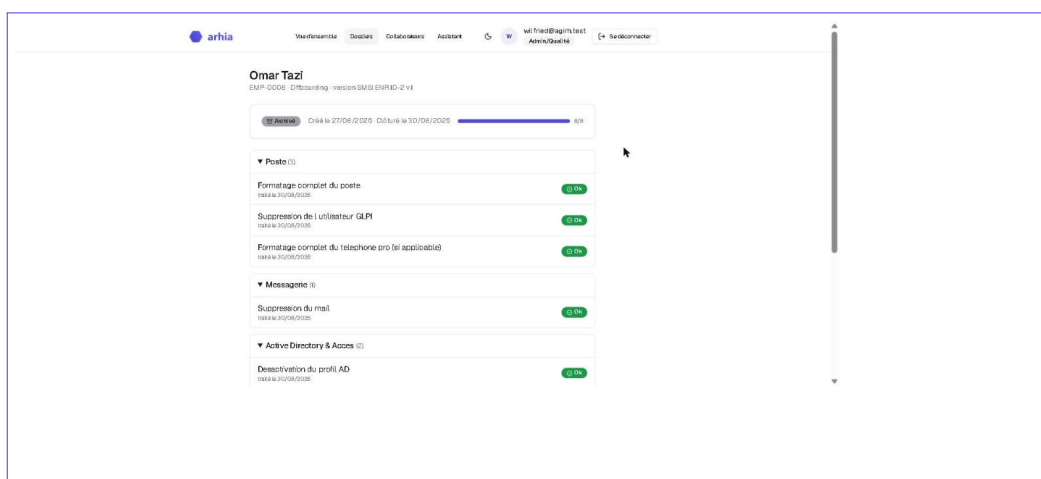
5.2 Dossiers et collaborateurs

Les pages Dossiers et Collaborateurs exposent, en lecture, l'ensemble des données déjà manipulables côté API : statuts de workflow (badges colorés distincts pour en cours, clôturé, archivé), progression de checklist, et types de contrat. Le détail d'un dossier affiche la checklist complète, groupée par section, avec le statut de chaque item et sa date de traitement.



Collaborateur	Type	Statut	Progression	Créé le	
Omar Tazi EMP-0008	Onboarding	Archivé	100%	27/06/2025	Voir
Nicolas Robin EMP-0003	Onboarding	En cours	0%	20/06/2025	Voir
Julie Monau EMP-0004	Onboarding	Archivé	100%	20/06/2025	Voir
Scotie Martin EMP-0002	Onboarding	En cours	0%	20/06/2025	Voir
Adam Perrot EMP-0005	Onboarding	En cours	0%	20/06/2025	Voir
Fatima Zahra Ahran EMP-0004	Onboarding	En cours	0%	20/06/2025	Voir
Nicolas Robin EMP-0003	Onboarding	En cours	0%	20/06/2025	Voir
Maryem Soukili EMP-0002	Onboarding	En cours	0%	20/06/2025	Voir
Hugo Chevillard EMP-0001	Onboarding	En cours	0%	20/06/2025	Voir

Liste des dossiers — statuts « Archivé » et « En cours » visibles simultanément, données réelles.



Omar Tazi
EMP-0008 - Onboarding - version SMIS EMP-02-V1

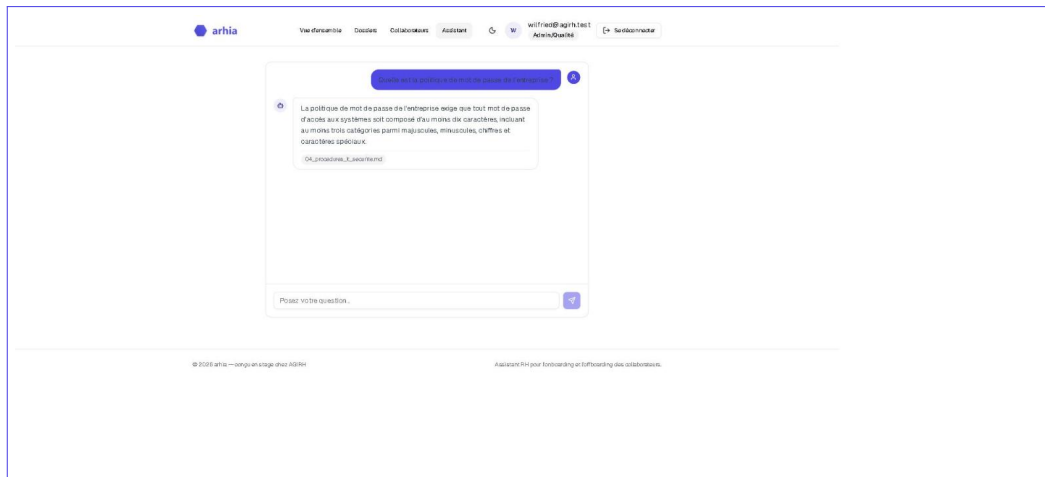
Statut: Archivé | Créé le 27/06/2025 | Dernière mise à jour 30/06/2025 | Progression: 8/8

- Poste (1)**
 - Formatage complet du poste: 100% (30/06/2025)
 - Suppression de l'unilatéral GLPI: 100% (30/06/2025)
 - Formatage complet du téléphone pro (si applicable): 100% (30/06/2025)
- Messagerie (1)**
 - Suppression du mail: 100% (30/06/2025)
- Active Directory & Accès (2)**
 - Désactivation du profil AD: 100% (30/06/2025)

Détail d'un dossier d'offboarding clôturé et archivé (8/8 items), avec la date de traitement de chaque item.

5.3 L'agent conversationnel en action

La page « Assistant » expose le cœur IA décrit au chapitre 3. La capture ci-dessous montre un échange réel : la question posée déclenche le pipeline complet (classification, recherche vectorielle, reranking, génération en streaming), et la réponse affichée cite explicitement le document source.



Échange réel avec l'agent : réponse sourcée sur 04_procedures_it_securite.md, conforme au contenu réel du document.

6. Résultats et vérifications

Une exigence a été tenue tout au long du stage : ne jamais considérer une fonctionnalité terminée sur la seule base d'une lecture de code, mais la vérifier en conditions réelles — HTTP réel, base de données réelle, modèle réellement invoqué — chaque fois que possible.

6.1 Tests automatisés

La suite de tests compte **245 tests** (xUnit, Moq, FluentAssertions) pour **0 warning** au build — 244 verts. Le seul échec est un flake pré-existant et documenté, `RagPipelineIntegrationTests` — accumulation de données dans la collection `Qdrant` persistante entre exécutions, sans rapport avec le code récent. Elle couvre la logique métier (RBAC, portée département, machine à états des templates), le pipeline RAG (chunking, tokenisation, reranking) et l'orchestration conversationnelle — notamment les cas de garde-fou (zéro candidat, tout sous le seuil de pertinence, dossier introuvable, portée refusée) — grâce à la testabilité permise par l'architecture hexagonale : aucun de ces tests ne démarre de base de données, de moteur ONNX ou de serveur Ollama réel.

6.2 Vérifications de bout en bout

Plusieurs garanties ont été vérifiées directement en HTTP, au-delà des tests unitaires : un RH tentant d'ouvrir directement l'URL d'un dossier hors de son pôle reçoit un refus explicite, jamais les données ; une question documentée produit une réponse sourcée, arrivant token par token en streaming ; une question hors périmètre est refusée poliment ; et un compte désactivé (`IsActive=false`) ne peut plus agir même avec un jeton encore valide, l'acteur étant relu en base à chaque requête plutôt que déduit uniquement du contenu du jeton.

6.3 Évaluation du pipeline RAG

Un jeu de 48 questions de référence (« gold »), rédigé spécifiquement pour ce projet et ancré sur le corpus réel — 36 questions documentées, 6 hors périmètre, 6 hors corpus — sert à mesurer objectivement le pipeline plutôt que de se fier à une impression qualitative. Isolée du reste du pipeline (sans appel LLM), l'étape de recherche documentaire seule atteint **36/36** : le problème mesuré au chapitre 3 ne vient donc pas de la recherche, mais de l'étape de classification d'intention en amont.

6.4 Déploiement

Le déploiement complet — quatre services orchestrés par Docker Compose, plus Ollama natif sur l'hôte, rejoint via `host.docker.internal` — a été vérifié sur une base fraîche : une seule commande démarre la pile, les migrations EF Core s'appliquent automatiquement, et un parcours complet (inscription, connexion, question sourcée) a été réalisé de bout en bout à travers toute la pile conteneurisée.

7. Difficultés rencontrées

Plusieurs difficultés techniques concrètes ont demandé un vrai travail d'investigation — retenues ici parce qu'elles ont été les plus formatrices.

7.1 Un décalage d'indices invisible entre deux tokenizers

Le modèle d'embedding attend des identifiants de tokens dans l'espace de convention Hugging Face, alors que le tokenizer SentencePiece officiel de Microsoft produit des identifiants dans son propre espace natif — un décalage qui ne se manifeste par **aucune erreur explicite** : le modèle produit des vecteurs, simplement incorrects, sans avertissement. Le diagnostic a exigé une comparaison ligne à ligne avec le fichier `tokenizer.json` officiel du modèle pour identifier précisément la correspondance attendue, avant d'écrire une fonction de correction dédiée. Cette classe de bug — un résultat plausible mais faux, sans signal d'erreur — s'est révélée la plus dangereuse rencontrée pendant le stage, précisément parce qu'elle ne se signale pas d'elle-même.

7.2 Une optimisation par défaut qui casse un flux progressif

Le streaming SSE s'est heurté, à deux niveaux indépendants de la pile, au même piège : une optimisation de bufferisation, pensée pour des réponses HTTP classiques, incompatible avec un flux progressif. Côté client .NET vers Ollama, l'absence du drapeau `HttpCompletionOption.ResponseHeadersRead` aurait fait attendre à `HttpClient` la réponse complète avant de la rendre disponible. Côté serveur Next.js, la compression HTTP intégrée bufferisait de la même manière toute la réponse avant envoi — corrigé par `compress: false` dans la configuration. Les deux pièges ont été trouvés empiriquement, en observant qu'aucun octet n'arrivait avant la fin complète du flux, plutôt que supposés à l'avance.

7.3 Un renommage de dossier refusé par Windows

Le renommage du produit (voir note en introduction) a exigé de renommer des dossiers de projet .NET récemment reconstruits. Windows a refusé le renommage (« Accès refusé ») sur plusieurs dossiers : la cause, trouvée par élimination, était des processus `dotnet.exe` de type serveur de compilation (MSBuild), laissés vivants en arrière-plan par une commande de build antérieure et tenant encore des fichiers ouverts. Un arrêt explicite de ces serveurs de compilation a résolu la majorité des cas ; pour le dossier restant, un contournement fiable a consisté à créer le nouveau dossier vide puis à déplacer chaque fichier individuellement plutôt que de renommer le dossier entier d'un bloc.

7.4 Vérifier une palette de couleurs plutôt que de la juger à l'œil

Lors du retravail de la palette sombre de l'interface (chapitre 5), un calcul programé du ratio de contraste WCAG — plutôt qu'une appréciation visuelle — a révélé que deux des six couleurs de statut existantes tombaient en dessous du seuil d'accessibilité AA (3,58 :1 et 3,95 :1 mesurés, contre 4,5 :1 requis), alors qu'elles paraissaient lisibles à l'œil sur l'écran de développement. Corrigées en assombrissant légèrement leur teinte de fond

jusqu'à franchir le seuil, elles ont été revérifiées par le calcul et en navigateur — une preuve concrète qu'une mesure objective détecte des problèmes qu'une simple relecture manque.

8. Compétences acquises

8.1 Compétences techniques

Ce stage a été l'occasion d'une montée en compétence concrète sur un pipeline RAG complet — du chunking structurel au reranking par cross-encodeur — au-delà d'un simple appel à une API d'embedding : comprendre pourquoi un bi-encodeur et un cross-encodeur ne jouent pas le même rôle, pourquoi une similarité cosinus exige une normalisation L2 en amont, ou pourquoi un score de similarité n'est pas une preuve de vérité, sont des enseignements qui dépassent ce seul projet. La mise en œuvre d'une architecture hexagonale à l'échelle réelle d'un projet complet — pas seulement sur un exemple d'école — a également été formatrice, en particulier sur la discipline qu'elle impose : résister à la tentation de faire fuiter une dépendance d'infrastructure dans la couche métier, même quand un raccourci semble plus rapide sur le moment.

8.2 Compétences méthodologiques

Le stage a imposé une discipline d'audit critique et continu du travail déjà produit — une compétence rarement mise en pratique aussi directement dans un cadre académique, où l'on cherche plus souvent à terminer un travail commencé qu'à le remettre en question dès qu'un problème structurel apparaît. La rigueur de mesure adoptée tout au long du projet — ne jamais affirmer un chiffre non mesuré, documenter une tentative abandonnée aussi honnêtement qu'une réussite — s'est imposée progressivement comme une méthode de travail à part entière, illustrée à plusieurs reprises dans ce rapport (chapitre 3, chapitre 7).

8.3 Compétences transversales

Le stage s'est déroulé avec une large autonomie de conception, contrebalancée par des points de décision explicites avec l'encadrant sur les choix engageants (périmètre, architecture, arbitrages de modèles) — un équilibre qui a exigé de savoir identifier, seul, quelles décisions relèvent effectivement d'un arbitrage produit à faire valider, plutôt que de tout remonter ou, à l'inverse, de tout trancher sans en référer.

9. Bilan et perspectives

9.1 Ce qui est livré et vérifié

- Un socle métier complet — RBAC à trois rôles, portée département, machine à états des templates, workflows d'onboarding et d'offboarding — testé et vérifié en conditions réelles.
- Un pipeline RAG à quatre phases obligatoires, de bout en bout en ONNX Runtime .NET pur, corpus réel indexé et interrogé avec succès.
- Une orchestration conversationnelle protégée par un garde-fou anti-hallucination écrit en code, couvert par des tests dédiés.
- Une interface complète par rôle — tableau de bord, dossiers, collaborateurs — au-delà du seul agent conversationnel, avec une administration dédiée pour la réindexation du corpus.
- Un frontend temps réel (chat et notifications SSE), en mode clair et sombre, avec une identité visuelle propre au produit.
- Un déploiement Docker Compose complet, vérifié sur base fraîche.

9.2 Limites assumées

Le taux de mauvais classement du routeur d'intention (environ 27 %) reste la limite la plus significative du système actuel, documentée et mesurée plutôt que masquée (chapitre 3). Trois cas particuliers métier — mutation inter-pôle, annulation ou suspension d'un dossier, pôle temporairement vacant — ont des propositions de comportement rédigées mais non encore validées en pratique. Les endpoints de lecture côté API existent désormais, mais certaines fonctionnalités d'administration (gestion des templates, gestion fine des comptes) restent à exposer côté interface.

9.3 Perspectives

Le levier le plus prometteur pour améliorer la fiabilité du routeur est le passage à des modèles plus capables sur un serveur dédié — une piste déjà préparée techniquement (chapitre 3) et à confirmer par une mesure réelle. Les phases restantes de l'interface (modèles de checklist, administration complémentaire, finitions du chat et des notifications) constituent la suite naturelle du travail déjà livré sur le frontend. Enfin, les trois cas particuliers métier identifiés en cadrage (chapitre 1) restent à implémenter une fois leur comportement précis validé avec l'entreprise.

Conclusion générale

Ce stage a permis de concevoir et de réaliser un système complet répondant à un besoin concret d'AGIRH : un agent conversationnel d'onboarding et d'offboarding appuyé sur un pipeline RAG rigoureux et une orchestration protégée par un garde-fou anti-hallucination vérifiable en code. Plus que n'importe quelle ligne de code produite, ce projet illustre l'approche adoptée tout au long de ce stage : vérifier plutôt que supposer, mesurer plutôt qu'affirmer, et documenter une limite aussi honnêtement qu'une réussite.

Le système livré reste un prototype : le taux de classification du routeur conversationnel et la couverture fonctionnelle du frontend définissent clairement ce qu'il reste à faire. Mais chaque brique livrée — du chunking structurel au circuit de validation qualité, en passant par le garde-fou anti-hallucination — l'a été vérifiée en conditions réelles, pas seulement conçue sur le papier. C'est, avec le recul, la contribution la plus durable de ce stage : au-delà du code lui-même, une méthode de travail rigoureuse, transférable à tout projet futur.

Références

1. Lewis, P. *et al.* (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Concept fondateur du RAG, à l'origine de l'architecture décrite au chapitre 3.
2. Cockburn, A. *Hexagonal Architecture (Ports and Adapters)*. Concept architectural à l'origine de la structure décrite au chapitre 2.
3. ISO/IEC 27001 — *Information security management systems*. Norme encadrant les checklists qualité réelles à l'origine du contenu métier (chapitre 1).
4. Documentation officielle ONNX Runtime — <https://onnxruntime.ai>.
5. Documentation officielle Qdrant — <https://qdrant.tech>.
6. Documentation officielle Ollama — <https://ollama.com>.
7. BAAI. *bge-reranker-v2-m3* — fiche modèle Hugging Face, modèle de reranking cross-encodeur utilisé (chapitre 3).
8. Documentation officielle Next.js — <https://nextjs.org/docs>.
9. Documentation officielle ASP.NET Core — <https://learn.microsoft.com/aspnet/core>.
10. Documentation officielle Microsoft.ML.Tokenizers — <https://learn.microsoft.com/dotnet/api/microsoft.ml.tokenizers>.

Annexes

Annexe A — Glossaire

Terme	Sens en une ligne
Architecture hexagonale	Domain (règles pures) → Core (use cases + ports) → Infrastructure (adaptateurs) → Api. Les dépendances pointent toujours vers l'intérieur.
RAG	Chercher l'information dans de vrais documents avant de la donner au LLM, plutôt que de compter sur sa mémoire.
Chunking	Découper un document en morceaux exploitables — ici, par structure, pas par taille fixe aveugle.
Embedding	Transformer du texte en vecteur numérique (ici 768 dimensions) pour comparer des sens, pas juste des mots.
ANN / HNSW	Recherche approximative des voisins les plus proches — pas exacte, mais rapide à grande échelle.
Similarité cosinus	Mesure d'angle entre deux vecteurs, bornée entre -1 et 1 grâce à la normalisation L2.
Bi-encodeur	Encode requête et document séparément — rapide, utilisé pour la recherche large.
Cross-encodeur / reranking	Encode requête et document ensemble, en une passe — lent, très précis, réservé à une courte liste de candidats.
Fail-safe (vs fail-open)	En cas de doute ou de panne, le système refuse — il n'accorde jamais un accès ou une réponse par défaut.
RBAC	Role-Based Access Control — qui a le droit de faire quoi, par rôle.
Scope guard / portée	Restreint une action à un périmètre précis (ici le département), vérifié séparément du RBAC, toujours après.
SSE	Server-Sent Events — flux HTTP unidirectionnel serveur vers client, utilisé pour le streaming du chat et des notifications.
BFF	Backend For Frontend — le frontend sert d'intermédiaire, le JWT n'est jamais exposé au navigateur.
Few-shot	Donner des exemples dans le prompt pour orienter le comportement du modèle, sans le réentraîner.
Garde-fou anti-hallucination	Mécanisme en code (pas une consigne de prompt) qui empêche une réponse non fondée.

Annexe B — Stack technique récapitulative

Composant	Choix
Backend	.NET 8, ASP.NET Core, EF Core
Frontend	Next.js 15 (BFF), TailwindCSS, shadcn/ui, react-markdown
Authentification	JWT + ASP.NET Identity
Temps réel	SSE
Base relationnelle	SQL Server (Docker)
Base vectorielle	Qdrant (Docker, ANN/HNSW, cosinus)
Chunking	Structurel + recouvrement, <code>Microsoft.ML.Tokenizers</code>
Embedding	ONNX Runtime .NET, multilingue 768d
Reranking	ONNX cross-encodeur BAAI/ <code>bge-reranker-v2-m3</code> (obligatoire)
LLM Router + Generator	Ollama local, <code>phi4-mini:3.8b</code>
Déploiement	Docker Compose local (4 services)
Tests	xUnit + Moq + FluentAssertions (245 tests, 244 verts + 1 flake connu)